

SystemVerilog Asynchronous FIFO Architecture Plan

<https://gemini.google.com/app/cff464351823e136>

User prompt: You are helping me develop a serious RTL/CDC portfolio project for digital VLSI / ASIC RTL roles. I want to build a parameterized asynchronous FIFO in SystemVerilog, but this is not intended to be a toy FIFO or a generic "async FIFO implementation" repository. The project should demonstrate genuine CDC understanding, SystemVerilog design fluency, rigorous verification, debugging ability, and quantitative engineering analysis. The final project must be strong enough to support technically defensible CV bullets and an interview discussion where I can explain the architecture, design decisions, verification methodology, bugs, and measured results in detail. PROJECT CONTEXT My career roadmap identifies an asynchronous FIFO with Gray-coded pointers and a CDC verification harness as the highest-priority missing project in my current RTL portfolio. The project is intended to close the following gaps: CDC and metastability-aware RTL design asynchronous FIFO architecture Gray-coded pointer synchronization 2-flop synchronizers full/empty generation across independent clock domains SystemVerilog design-subset fluency stronger verification methodology CDC failure analysis synthesis/timing/resource awareness The roadmap specifically recommends that the project go beyond simply implementing the FIFO. The impressive version should include: Gray-code pointers 2FF synchronization self-checking verification rigorous randomized testing deliberately broken CDC implementations waveform evidence of failures CDC review/checklist documentation measurable results The project should therefore be designed around those requirements from the beginning. IMPORTANT WORKFLOW You are allowed and encouraged to: propose architectures make sophisticated implementation decisions suggest alternative microarchitectures choose appropriate SystemVerilog constructs propose verification strategies identify useful metrics suggest optimizations recommend tooling generate candidate RTL generate testbench code suggest assertions analyze failures suggest improvements Do not artificially simplify the project merely to make it easier. I want to understand the choices you make. I will read and interrogate the generated code, ask questions about anything I do not understand, challenge architectural decisions where appropriate, and may modify your proposals. The final code will be manually rewritten by me line-by-line after I understand it. Therefore, optimize for technical quality, explainability, and interesting engineering decisions rather than deliberately avoiding sophisticated techniques. Do not assume that "simple is always better." If a clever or more advanced decision is technically justified, propose it and explain why it is useful. PROJECT OBJECTIVES The final project should demonstrate: SystemVerilog RTL design CDC-safe architecture Metastability-aware reasoning Gray-code pointer synchronization Dual-clock FIFO operation Self-checking verification Randomized stress testing Assertion-based verification CDC fault injection Waveform-based debugging Parameterization Synthesis/resource analysis Timing analysis Quantitative verification metrics Reproducible regressions Engineering documentation The project should not be considered complete merely because the FIFO "works." The final repository should contain enough evidence to answer: How does the FIFO work? Why is it CDC-safe? Why are Gray-coded pointers needed? Why is a binary pointer unsafe to cross directly? How are metastability risks handled? How are full and empty detected? How was correctness verified? How aggressively was it tested? What happens when the CDC architecture is intentionally broken? What bugs were found? What does synthesis say about the implementation? What are the timing/resource characteristics? How does the design scale with width and depth? What quantitative evidence supports the CV claims? TARGET ARCHITECTURE Implement a true asynchronous FIFO with: independent write clock independent read clock parameterized data width parameterized address width FIFO depth = $2^{\text{ADDR_WIDTH}}$ independent read/write pointer domains binary pointers Gray-coded pointers synchronized Gray pointers crossing clock domains 2-flop synchronizers CDC-aware full generation CDC-aware empty generation parameterized storage valid write/read handshaking clear behavior for full and empty conditions Suggested starting configuration: DATA_WIDTH = 32 ADDR_WIDTH = 4 This gives a 16-entry FIFO with 32-bit data. However, the implementation should remain genuinely parameterized rather than being specialized around this one configuration. ARCHITECTURAL ANALYSIS Before writing the final implementation, explain and establish the architecture in detail. Cover: Clock domains Write-side state Read-side state Pointer representation Binary-to-Gray conversion Gray-to-binary conversion if needed Pointer width FIFO addressing CDC crossings Synchronizer placement Full detection Empty detection Memory organization Reset behavior Assumptions about simultaneous reads/writes Throughput behavior Latency implications Parameter scaling Derive the pointer-width requirement. Explain why an asynchronous FIFO normally uses ADDR_WIDTH+1 pointer bits even though the memory address uses ADDR_WIDTH bits. Derive the Gray-pointer full and empty conditions rather than presenting them as unexplained formulas. For the full detector, explicitly explain why the synchronized read Gray pointer must be transformed before comparison and how that relates to the wraparound representation. For the empty detector, explain why the synchronized write pointer is compared against the next read pointer. Explain precisely what crosses the CDC boundary and what does not. SYSTEMVERILOG DESIGN Use SystemVerilog rather than Verilog. Use the language features that materially improve the design. Potentially relevant constructs include: logic always_ff always_comb typedef enum packed arrays unpacked arrays parameters localparam generate functions packages interfaces/modports where appropriate assertions concurrent SVA appropriate SystemVerilog typing and compile-time checks Choose the constructs you believe are technically appropriate and explain those choices. Do not artificially restrict yourself to a predetermined subset of SystemVerilog. At the

same time, keep the implementation synthesizable and appropriate for RTL/ASIC design. Suggested source hierarchy: rtl/ async_fifo.sv fifo_mem.sv cdc_sync.sv bin2gray.sv gray2bin.sv The exact hierarchy may be changed if you determine a better organization. CDC SYNCHRONIZER DESIGN Create a reusable CDC synchronization module. Requirements: parameterizable signal width destination clock reset handling two synchronization stages clean destination-domain output Explain: metastability what happens at the first synchronizer stage what the second stage accomplishes why synchronizer MTBF improves with additional resolution time why simulation cannot literally reproduce analog metastability behavior why the FIFO's pointer encoding matters before synchronization even occurs Consider whether the implementation should expose attributes or synthesis hints commonly used for synchronizer recognition, and explain the implications of any such decision. GRAY CODE Implement Gray-code conversion cleanly. Provide: binary -> Gray Gray -> binary if required by the architecture Verify: mathematical correctness exhaustive conversion for small widths round-trip correctness Gray transition behavior Explicitly verify that sequential pointer transitions have a Hamming distance of one in the Gray representation. Record quantitative results: number of transitions tested number expected to satisfy the one-bit property number actually satisfying it number of violations PARAMETERIZATION The design must be parameterized. At minimum: DATA_WIDTH ADDR_WIDTH The test and synthesis flow should exercise multiple parameter configurations. Suggested configurations: DATA_WIDTH=8, ADDR_WIDTH=2, depth=4 DATA_WIDTH=16, ADDR_WIDTH=3, depth=8 DATA_WIDTH=32, ADDR_WIDTH=4, depth=16 DATA_WIDTH=64, ADDR_WIDTH=5, depth=32 You may add larger configurations if simulation/synthesis time remains reasonable. Quantify: number of configurations tested smallest data width largest data width smallest depth largest depth total parameter combinations tested number passed number failed The actual final values must come from experiments. FUNCTIONAL TESTBENCH Build a self-checking testbench. The testbench should include a reference model / scoreboard rather than relying only on manual waveform inspection. The scoreboard should model FIFO ordering and occupancy independently from the RTL implementation. Track: writes attempted writes accepted writes rejected because full reads attempted reads accepted reads rejected because empty data comparisons mismatches current reference occupancy maximum observed occupancy minimum observed occupancy The testbench should detect: data loss data duplication data corruption ordering errors illegal reads illegal writes pointer/flag inconsistencies CLOCK-DOMAIN STRESS Do not assume related clocks. Run independent write/read clocks. At minimum exercise several clock-period combinations with different ratios and phase relationships. For example: 5 ns / 11 ns 7 ns / 11 ns 7 ns / 13 ns 10 ns / 20 ns 11 ns / 7 ns 13 ns / 7 ns 20 ns / 10 ns You may choose a better set or add more configurations. The objective is not to collect arbitrary combinations but to exercise: write faster than read read faster than write roughly balanced clocks awkward phase relationships nontrivial frequency ratios Quantify: number of clock configurations number passed number failed simulation cycles per configuration total cycles across the regression RANDOMIZED STRESS TESTING The testbench should support randomized traffic. Randomize where appropriate: write enable read enable write data burst lengths idle periods clock ratios traffic intensity test duration reset timing where the reset specification permits it Include distinct traffic profiles: write-heavy read-heavy balanced alternating burst write / burst read continuous traffic long idle periods repeated fill/drain cycles sustained near-full operation sustained near-empty operation Use deterministic random seeds and always record the seed. The regression must be reproducible. QUANTITATIVE VERIFICATION TARGETS Design the environment so that useful statistics are automatically collected. Initial target: at least 100,000 attempted FIFO operations at least 50,000 accepted operations multiple independent clock configurations multiple FIFO parameter configurations Stretch target: approximately 1,000,000 or more attempted operations across the full regression The actual number should be selected based on simulation runtime and confidence, then reported exactly. Do not invent a number for the CV. Collect: total simulation cycles total simulation time attempted writes accepted writes rejected writes attempted reads accepted reads rejected reads data comparisons mismatches assertion failures random seeds testcases executed testcases passed testcases failed ASSERTION-BASED VERIFICATION Use SystemVerilog assertions to capture meaningful invariants. At minimum consider properties for: No write acceptance while full No read acceptance while empty Valid pointer update conditions Gray pointer transition properties Reset initialization FIFO occupancy bounds Data ordering Correct full/empty relationships Synchronizer behavior Pointer stability where applicable Choose additional properties where you believe they strengthen the project. Separate: immediate assertions concurrent assertions simulation checks Track actual assertion results. Quantify: number of assertion properties number of assertion evaluations if practical failures unexpected failures passes where the simulator provides useful counts The target is zero unexpected assertion failures in the correct implementation. BROKEN CDC DESIGN #1 — BINARY POINTER CROSSING Create an intentionally broken version of the FIFO in which an ordinary binary pointer is crossed between clock domains instead of Gray coding the pointer appropriately. The implementation should remain syntactically and structurally plausible. Run the same verification environment against it. The objective is to demonstrate: why the architecture is incorrect what observable failures can result how the failure manifests what waveform evidence looks like how the correct architecture prevents the underlying problem Do not falsely claim that RTL simulation reproduces physical metastability. Instead distinguish: architectural CDC unsafety simulator-visible functional consequences physical metastability risk in real silicon Capture: test configuration seed first failure failing transaction expected value actual value relevant pointer values synchronized values waveform root cause BROKEN CDC DESIGN #2 — SYNCHRONIZER FAILURE Create another intentionally broken version. Potential examples: missing synchronization stage improperly synchronized pointer incorrect synchronizer connection combinational logic inserted into an unsafe location another CDC mistake you judge to be more educational Choose a defect that teaches something meaningfully different from the first broken implementation.

Run the same regression. Document the failure and root cause. **BUG/FAILURE POSTMORTEM** Create a bug log. Capture real bugs encountered during development, plus controlled injected bugs where necessary. For every significant bug document: symptom test that exposed it failing seed waveform timestamp relevant state/signals root cause why the bug occurred why the original implementation allowed it correction regression added to prevent recurrence **Potential deliberately injected bugs:** incorrect full detection incorrect empty detection wrong Gray conversion wrong pointer width bad synchronizer binary pointer CDC pointer increment under the wrong condition reset sequencing issue address extraction bug off-by-one occupancy behavior Do not manufacture fake "bugs found during development." Distinguish real bugs from deliberately injected validation defects. **CDC ANALYSIS DOCUMENT** Create: docs/cdc_analysis.md Include: Clock-domain inventory All signals crossing between domains Source domain Destination domain Signal width Synchronization method CDC rationale Metastability discussion Gray-code rationale Full/empty flag rationale Multi-bit CDC considerations Reconvergence considerations Reset-domain considerations Potential CDC violations Why the final architecture is structurally sound Include a table: Signal | Source Domain | Destination Domain | Width | Synchronizer | CDC Rationale Make this document useful as an interview artifact. **CDC CHECKLIST** Create a concise review checklist covering items such as: Are all asynchronous control signals synchronized? Are multi-bit CDC signals handled correctly? Are Gray-coded counters used where appropriate? Are synchronizer stages placed in the destination domain? Are synchronizer outputs used rather than asynchronous inputs? Are CDC reconvergence hazards considered? Are resets handled consistently? Are full/empty conditions generated entirely within their owning clock domains? Are there any combinational paths crossing clock domains? Are there assumptions about clock frequency or phase that are not actually guaranteed? Are parameter changes still CDC-safe? **SYNTHESIS** After RTL simulation is stable, synthesize the implementation. Use an accessible open-source synthesis flow such as Yosys unless another local tool is more appropriate. Record actual measured metrics. Potential metrics: total cell count flip-flop count combinational logic count memory implementation estimated area timing delay maximum frequency critical path logic depth warnings inferred latches if any inferred memories other useful synthesis statistics Distinguish clearly between: ASIC synthesis metrics FPGA synthesis metrics Do not represent FPGA LUT/FF counts as ASIC area. If FPGA synthesis is also run, record: LUTs FFs BRAM DSP usage utilization Fmax timing slack **TIMING ANALYSIS** Investigate the critical path. Identify: startpoint endpoint path delay combinational depth limiting logic slack where available maximum frequency where available Use the results to discuss how the RTL architecture affects timing. Do not optimize blindly for Fmax. Explain the tradeoffs. If a timing constraint flow is available, generate appropriate constraints for the independent clocks and explain the CDC-related timing assumptions. **RESOURCE SCALING** Use parameter sweeps where practical. For multiple DATA_WIDTH and ADDR_WIDTH values, measure how: storage size register count logic count area proxy timing resource usage scale with the design parameters. This should produce useful graphs or tables rather than merely claiming "parameterized." For example, potentially produce: Resource count vs DATA_WIDTH Resource count vs FIFO depth Estimated memory bits vs FIFO depth Fmax vs DATA_WIDTH Fmax vs FIFO depth Only generate plots that are meaningful for the chosen implementation/tool. **AUTOMATED METRICS** Create scripts capable of collecting the final metrics automatically. Potentially include: scripts/run_sim.sh scripts/run_regression.sh scripts/synth.sh scripts/collect_metrics.py The exact scripts are up to you. The key requirement is reproducibility. A single regression command should ideally produce: pass/fail summary transaction counts assertion results random seeds clock configurations parameter configurations A synthesis command should produce: synthesis report timing report where available resource report The metrics parser should convert those reports into a concise machine-readable summary such as CSV/JSON/TXT if useful. **FINAL RESULTS TABLE** Create a final project-results document. Include measured values for: Design: data width FIFO depth pointer width number of clock domains number of CDC crossings number of synchronizer stages Verification: total attempted operations accepted writes accepted reads blocked writes blocked reads data comparisons mismatches assertion properties assertion failures clock configurations parameter configurations total simulation cycles total regression runtime random seeds CDC: Gray transitions checked Gray violations intentionally broken designs broken-design failures detected Hardware: register count logic/cell count memory bits area/resource estimate critical path Fmax timing slack where available Do not fabricate values. Do not round metrics upward just to make the CV sound better. **CV METRIC STRATEGY** The final CV should focus on a few particularly strong numbers. The most useful categories are: Verification scale e.g. number of randomized operations CDC stress e.g. number of independent clock-ratio configurations Parameterization e.g. number/range of FIFO configurations Correctness e.g. zero data mismatches / zero assertion failures CDC fault analysis e.g. number of deliberately broken architectures successfully detected and diagnosed Hardware metrics e.g. register count, area/resource usage, Fmax, timing slack Scaling e.g. validated data widths and FIFO depths Potential CV bullet structures: "Designed a parameterized dual-clock asynchronous FIFO in SystemVerilog using Gray-coded pointers and 2FF CDC synchronization; stress-tested across X clock-ratio configurations and Y randomized operations with 0 data mismatches." "Developed a self-checking CDC verification environment with scoreboard and SVA; validated Z parameter configurations, exercised N functional scenarios, and diagnosed M intentionally broken CDC architectures through waveform analysis." "Synthesized the parameterized FIFO using TOOL and measured X registers, Y area/resource usage, and Z MHz Fmax at the selected target." These examples are structures only. Replace every placeholder with actual measured values. Do not force every available metric onto the CV. Select the metrics that communicate scale and engineering quality most efficiently. **THROUGHPUT METRIC** Do not automatically report a generic "FIFO throughput" number. Throughput depends on: write clock frequency read clock frequency interface assumptions traffic pattern occupancy backpressure If throughput is measured, define precisely what

is being measured. For example: accepted writes per second accepted reads per second sustained throughput under a specified clock configuration Only report throughput when the measurement is clearly defined and meaningful. WAVEFORM ARTIFACTS Generate a small set of high-value waveform examples rather than dumping arbitrary waveforms into the repository. At minimum capture: Reset -> empty Normal write/read FIFO filling FIFO reaching full FIFO draining Pointer wraparound Gray-code transition Correct CDC synchronization Broken binary-pointer CDC failure Second broken CDC failure Annotate screenshots where useful. README Create a strong technical README. Suggested structure: Project overview Motivation Architecture CDC architecture Gray-code explanation Full/empty detection SystemVerilog design Verification architecture Randomized testing Assertions Deliberately broken CDC versions Waveform evidence Quantitative verification results Synthesis results Timing results Parameter scaling Bug postmortem Lessons learned Limitations Future improvements Include a clean architecture diagram. Include a clock-domain diagram. Include a compact quantitative-results table near the top of the README so the project's scale is immediately visible. TOOLS The primary development flow should work with open tools available locally. Potential tools: SystemVerilog Icarus Verilog Verilator GTKWave Yosys Python Bash Make Use whichever combination provides the best workflow. Commercial EDA tool knowledge is not required for this project. Where practical, explain how the open-source tools correspond conceptually to commercial RTL/simulation/synthesis tools. VERIFICATION PHILOSOPHY The correct FIFO should pass its complete regression. The broken implementations should be demonstrably exposed by the verification system where the particular defect is observable. The verification environment should not merely be a stimulus generator. It should provide: reference modeling checking assertions reproducibility regression statistics failure localization waveform generation quantitative reporting IMPORTANT DISTINCTION ABOUT METASTABILITY Do not describe RTL simulation as proof that metastability is impossible. The project should explicitly distinguish: RTL correctness CDC architectural correctness metastability probability / MTBF analog physical behavior The objective is to demonstrate that the design uses the established architectural techniques intended to make asynchronous-domain communication robust. FINAL QUALITY BAR The project should be considered complete only when: The final implementation is SystemVerilog. The FIFO is genuinely parameterized. The design uses an appropriate asynchronous FIFO architecture. Binary and Gray pointers are correctly implemented. CDC synchronization is correctly implemented. Full/empty logic is verified. The testbench is self-checking. Randomized regressions are reproducible. Multiple clock relationships are tested. Multiple parameter configurations are tested. Assertions are present and passing. Gray-code behavior is explicitly verified. At least two intentionally broken CDC architectures are created. Broken designs produce meaningful diagnostic evidence where expected. At least one CDC failure is demonstrated in a waveform. Real development bugs are documented. Regression statistics are automatically collected. Synthesis metrics are collected. Timing information is collected where practical. Resource/parameter scaling is analyzed where practical. The CDC crossings are documented. The README contains quantitative results. The final metrics are reproducible from scripts. The final CV claims can be traced directly to project artifacts. QUANTIFICATION MUST BE A DESIGN REQUIREMENT, NOT A FINAL FORMATTING STEP Do not wait until the end to ask, "What numbers can we put on the CV?" Instead, design the project so meaningful measurements naturally emerge from the work. The final project should ideally allow statements such as: "verified across N independent clock configurations" "stress-tested with N randomized operations" "validated across N parameter configurations" "performed N data comparisons with 0 mismatches" "verified N assertions with 0 unexpected failures" "checked N Gray-pointer transitions with 0 violations" "diagnosed N intentionally broken CDC implementations" "synthesized to N registers / N logic elements / N area" "achieved N MHz estimated Fmax" "validated data widths from X to Y bits" "validated FIFO depths from X to Y entries" Only use statements that the actual project data supports. THE FINAL PROJECT SHOULD COMMUNICATE THREE THINGS I can design RTL. I can verify and debug RTL. I can quantify the behavior and implementation of what I designed. The project should therefore be developed as an engineering experiment and portfolio artifact, not simply as a coding exercise. STARTING TASK Before generating the complete codebase: Propose the detailed architecture. Explain the key design decisions. Identify the CDC crossings. Explain the pointer/full/empty mathematics. Propose the repository structure. Propose the verification architecture. Propose the quantitative metrics and how each will be measured. Propose the deliberate CDC faults. Propose the synthesis/timing experiment plan. Identify any architectural decisions that require my approval before implementation. Then proceed incrementally through implementation, verification, debugging, and quantitative analysis. At every major stage, explain what is being built, why it is built that way, what should be measured, and what evidence should eventually appear in the repository and CV. Proceed with the project, one file at a time.

Response: 1. System Architecture & Block Partitioning

The architecture uses the Clifford Cummings dual-clock asynchronous FIFO topology, decoupled into five dedicated SystemVerilog modules. This partitioning isolates clock domains, keeps CDC synchronizers distinct from control logic, and prevents synthesis tools from optimizing across clock boundaries.

- `rp_ptr_empty.sv`: Houses read pointer generation (binary and Gray), address calculation (`raddr`), and pessimistic `empty` assertion via look-ahead Gray comparison.
- `cdc_sync.sv`: Generic parameterizable multi-bit 2-flop synchronizer with synthesis attributes (`dont_touch`, `async_reg` = "TRUE") to prevent shift-register LUT (SRL) inference and ensure tight placement.

2. Clock Domains & Signal Crossing Inventory

The FIFO contains two asynchronous clock domains: Write Clock (`wclk`) and Read Clock (`rclk`). No static phase or frequency relationship is assumed.

Signal Name	Source Domain	Destination Domain	Width	Synchronization Method	CDC Rationale / Constraint
<code>wptr_gray</code>	<code>wclk</code>	<code>rclk</code>	<code>ADDR_WIDTH + 1</code>	2-stage DFF (<code>cdc_sync</code>)	Gray-coded: exactly 1 bit transitions per increment, eliminating multi-bit bus skew hazards during sampling.
<code>rp_ptr_gray</code>	<code>rclk</code>	<code>wclk</code>	<code>ADDR_WIDTH + 1</code>	2-stage DFF (<code>cdc_sync</code>)	Gray-coded: safe against intermediate bus states.
<code>wdata / waddr</code>	<code>wclk</code>	Memory core → <code>rclk</code>	<code>DATA_WIDTH / ADDR_WIDTH</code>	Data-path qualification by pointer separation	Memory locations are guaranteed not to be concurrently written and read; pointers ensure full/empty boundaries enforce safe setup/hold margins.
<code>wrst_n</code>	External	<code>wclk</code>	1	Asynchronous assert, synchronous deassert bridge	Prevents reset removal timing violations in the write domain.
<code>rrst_n</code>	External	<code>rclk</code>	1	Asynchronous assert, synchronous deassert bridge	Prevents reset removal timing violations in the read domain.

3. Pointer Width & Full/Empty Mathematical Derivations

Pointer Width Requirement: Why $N + 1$ Bits for 2^N Depth

For a FIFO of depth $D = 2^N$, the memory address requires $N = \text{ADDR_WIDTH}$ bits (0 to $2^N - 1$). However, a pointer must track both the current offset within the circular buffer and the number of times the pointer has wrapped around the buffer boundaries.

If an N -bit pointer is used, when the write pointer equals the read pointer (`wptr == rp_ptr`), the FIFO state is ambiguous:

- The FIFO could be completely **empty** (0 items; write pointer has written nothing or written and read at identical rates).
- The FIFO could be completely **full** (2^N items; write pointer has wrapped around exactly once and caught up to the read pointer from behind).

By extending the pointer to $(N + 1)$ bits:

- The lower N bits ($\text{pointer}[N-1:0]$) index the physical RAM address ($\text{addr} = \text{pointer}[N-1:0]$).
- The $(N + 1)$ -th bit ($\text{pointer}[N]$, the MSB) represents the **wrap bit** (parity of complete buffer cycles).

$$\text{FIFO Occupancy} = (wptr_{\text{bin}} - rptr_{\text{bin}}) \pmod{2^{N+1}}$$

- **Empty State (Binary):** Pointers are identical in all $(N + 1)$ bits:

$$wptr_{\text{bin}}[N : 0] == rptr_{\text{bin}}[N : 0]$$

- **Full State (Binary):** Pointers have identical lower N bits (same memory location), but inverted MSBs (write pointer has completed one more traversal than the read pointer):

$$wptr_{\text{bin}}[N] \neq rptr_{\text{bin}}[N] \quad \wedge \quad wptr_{\text{bin}}[N - 1 : 0] == rptr_{\text{bin}}[N - 1 : 0]$$

Binary-to-Gray Conversion & Single-Bit Transition Invariant

Binary counters cannot be directly synchronized across clock domains because incrementing a binary vector often flips multiple bits simultaneously (e.g., $0111_2 \rightarrow 1000_2$ flips 4 bits). Propagation skew across physical wires causes the destination domain to sample transient non-sequential states (e.g., 0000_2 or 1111_2), causing catastrophic false full/empty assertions.

Gray coding ensures that between any two consecutive values k and $k + 1$, exactly **one** bit changes:

$$G[i] = B[i] \oplus B[i + 1] \quad \text{for } 0 \leq i < N, \quad G[N] = B[N]$$

In SystemVerilog:

```
assign gray_ptr = bin_ptr ^ (bin_ptr >> 1);
```

Because depth is 2^N , the standard $(N + 1)$ -bit Gray code wraps from $2^{N+1} - 1$ back to 0 with a single bit transition, preserving the Gray property across circular wraparounds.

Derivation of the Gray-Code Empty Condition

In the read domain, the FIFO is empty when the next read pointer catches up to the synchronized write pointer.

In binary:

$$rptr_{\text{bin}} == wptr_{\text{bin}}$$

Because binary-to-Gray conversion is a bijective (one-to-one) function, $B_1 = B_2 \iff G(B_1) = G(B_2)$. Therefore, the condition in Gray code is an exact, bitwise equivalence:

$$\text{rempty_val} = (rptr_gray_next == wptr_gray_sync)$$

Conservative Nature: Because $wptr_gray_sync$ passes through a 2-stage synchronizer, the read domain sees the write pointer delayed by up to 2 `rc1k` edges. If writes occur, the read domain may perceive the FIFO as empty longer than it physically is. This latency is 100% safe: it prevents underflow, affecting only read latency.

Derivation of the Gray-Code Full Condition

In binary, the full condition requires $wptr_{\text{bin}}$ to be ahead of $rptr_{\text{bin}}$ by exactly 2^N entries:

$$wptr_{\text{bin}} = rptr_{\text{bin}} + 2^N$$

Let B be the $(N + 1)$ -bit binary read pointer $B = [b_N, b_{N-1}, b_{N-2}, \dots, b_0]$. Adding 2^N flips the MSB b_N , leaving all lower bits unchanged:

$$B' = B + 2^N = [\sim b_N, b_{N-1}, b_{N-2}, \dots, b_0]$$

Now compute Gray code $G(B)$ and $G(B')$ bit by bit using $G[i] = B[i] \oplus B[i + 1]$:

1. For the MSB ($i = N$):

$$G'[N] = B'[N] = \sim b_N = \sim G[N]$$

(The MSB is inverted).

2. For bit $N - 1$ ($i = N - 1$):

$$G'[N - 1] = B'[N - 1] \oplus B'[N] = b_{N-1} \oplus (\sim b_N) = \sim (b_{N-1} \oplus b_N) = \sim G[N - 1]$$

(Bit $N - 1$ is also inverted).

3. For all lower bits ($i < N - 1$):

$$G'[i] = B'[i] \oplus B'[i + 1] = b_i \oplus b_{i+1} = G[i]$$

(All bits from $N - 2$ down to 0 remain strictly identical).

Thus, the full condition in $(N + 1)$ -bit Gray code requires inverting the top two bits of the synchronized read pointer while keeping the remaining bits equal:

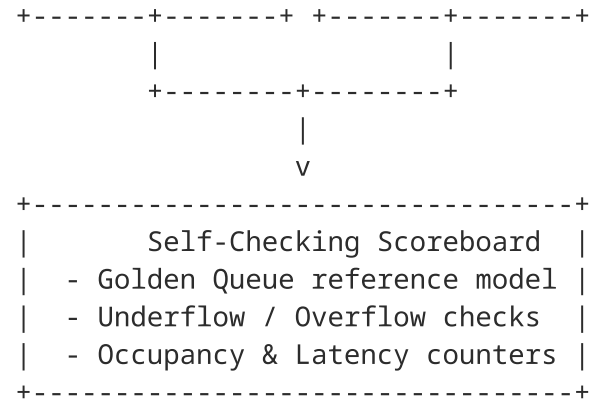
$$\mathbf{wfull_val} = (wptr_gray_next == \{\sim rp_gray_sync[N : N - 1], rp_gray_sync[N - 2 : 0]\})$$

```
assign wfull_val = (wptr_gray_next == {~rp_ptr_gray_sync[ADDR_WIDTH:ADDR_WIDTH-1],
                                     rp_ptr_gray_sync[ADDR_WIDTH-2:0]});
```

Conservative Nature: *rp_ptr_gray_sync* is delayed by 2 *wclk* cycles. The write domain may perceive the FIFO as full longer than it physically is, preventing overflow and guaranteeing data integrity.

4. Proposed Repository Structure

```
async_fifo_core/
├── rtl/
│   ├── async_fifo.sv      # Top-level parameterized structural wrapper
│   ├── fifo_mem.sv       # Dual-port parameterized SRAM storage array
│   ├── wptr_full.sv      # Write pointer management & full detection
│   ├── rptr_empty.sv     # Read pointer management & empty detection
│   └── cdc_sync.sv       # Parameterized 2-flop synchronizer with constraints
├── fault_injection/
│   ├── broken_binary_cdc.sv # CDC Bug #1: Direct binary pointer crossing
│   └── broken_sync_cdc.sv  # CDC Bug #2: 1-stage sync / combinational input glitch
├── tb/
│   ├── pkg/
│   │   ├── fifo_types_pkg.sv # Types, configuration structs, logging macros
│   │   └── fifo_agent_pkg.sv # Scoreboard, driver, monitor, generator classes
│   ├── sva/
│   │   ├── async_fifo_sva.sv # Concurrent assertions (SVA) bound to FIFO
│   │   └── gray_chk_sva.sv   # Formal assertion of Hamming distance == 1
│   ├── tb_async_fifo.sv     # Top-level testbench with dynamic clock generators
│   └── tb_fault_injector.sv # Testbench running fault-injection experiments
├── sim/
│   ├── Makefile            # Simulation targets for Icarus, Verilator, Questa/VCS
│   └── waves.gtkw          # GTKWave configuration for CDC debug
├── synth/
│   ├── synth_yosys.tcl     # Yosys synthesis script for gate-level/cell-level stats
│   └── opensta.sdc         # SDC timing constraints (set_clock_groups -asynchronous)
```

Test Traffic Profiles

1. **Burst Write / Burst Read:** Fill to full, hold, drain to empty. Tests flag boundary transitions.
2. **Alternating 1:1 Push-Pop:** Continuous steady-state throughput.
3. **Write-Heavy / Read-Starved:** Sustained full-backpressure testing.
4. **Read-Heavy / Write-Starved:** Sustained empty-backpressure testing.
5. **Randomized Stress:** Fully independent random delays, transaction bursts, and reset assertions.
6. **Clock Domain Sweeps:** Exhaustive matrix of clock frequency ratios:
 - Fast Write, Slow Read (100 MHz : 10 MHz)
 - Slow Write, Fast Read (10 MHz : 100 MHz)
 - Near-Equal Asynchronous Clocks (100 MHz : 99.5 MHz — maximizes metastability/edge collision windows)
 - High-Ratio Prime Clocks (133 MHz : 37 MHz)

Assertion-Based Verification (SVA)

- `assert_no_write_when_full: winc && wfull |-> ##0 !written`
- `assert_no_read_when_empty: rinc && rempty |-> ##0 !data_valid`

- `assert_gray_single_bit_flip`: For every pointer increment, $\text{\$countones}(\text{ptr_gray} \wedge \text{\$past}(\text{ptr_gray})) == 1$
- `assert_reset_values`: Immediate check that pointers and flags reset to 0/1 within bounded cycles.
- `assert_fifo_occupancy_bounds`: Scoreboard checks $0 \leq \text{occupancy} \leq 2^N$.

6. Deliberate CDC Fault Injection Plan

Broken CDC Design #1: Direct Binary Pointer Crossing (`broken_binary_cdc.sv`)

- **Mechanism:** Bypasses Gray coding entirely; registers binary write/read counters and passes them directly through 2-flop synchronizers.
- **Failure Mode Exposed:** Under asynchronous clock ratios with random phase drift, multi-bit transitions ($0111_2 \rightarrow 1000_2$) are sampled with physical/simulated bit-skews. The destination domain samples intermediate values (0000_2 , 1111_2 , etc.).
- **Observable Symptom:** The empty/full flag miscalculates occupancy, causing the testbench scoreboard to detect data corruption (reading stale entries or overwriting unread data) or spontaneous deadlocks.
- **Demonstrated Takeaway:** Explains why Gray coding is mathematically mandatory for multi-bit counters crossing asynchronous boundaries.

Broken CDC Design #2: Missing 2nd Sync Stage / Combinational Glitch Crossing (`broken_sync_cdc.sv`)

- **Mechanism:** Passes an un-registered combinational signal into the destination domain, or uses a single-flop synchronizer combined with simulated setup/hold non-deterministic jitter injection.
- **Failure Mode Exposed:** Combinational logic contains static and dynamic hazard glitches. Sampling combinational logic directly across a clock boundary synchronizes intermediate glitches into valid logic transitions, corrupting the synchronized pointer value.
- **Observable Symptom:** Sudden multi-step pointer jumps in the destination domain, triggering assertion failures in `gray_chk_sva`.

7. Synthesis, Timing & Resource Parameter Sweeps

Using **Yosys** and open cell libraries (e.g., Nangate 45nm or generic CMOS targets):

1. Parameter Sweeps:

- Sweep `DATA_WIDTH` across $[8, 16, 32, 64, 128]$ bits.
- Sweep `ADDR_WIDTH` across $[2, 3, 4, 5, 6, 7]$ (Depths 4 to 128).

2. Metrics Collected Automatically:

- Total DFF Count vs. Combinational Gate Count.
- Memory implementation style (Flip-flop register array vs. latch/macro cells).
- Critical Path Analysis: Identifies limiting paths ($wptr_{bin} \rightarrow \text{Adder} \rightarrow \text{Bin2Gray} \rightarrow \text{Lookahead Comparator} \rightarrow wfull$).
- F_{max} in write and read domains under independent constraints.

8. Architectural Decisions for Confirmation

1. Look-Ahead Registered Flags vs. Combinational Outputs:

- *Proposed:* Full and empty flags are computed combinationaly using the *next* pointer value (`wptr_gray_next`) and immediately registered into dedicated `wfull` and `rempty` flip-flops. This ensures zero output delay and clean glitch-free outputs at the cost of one extra comparator in the critical path.

2. Memory Array Model:

- *Proposed:* Write-enable gated dual-port register array inside `fifo_mem.sv` that infers standard FPGA block RAMs or maps cleanly to ASIC register files without combinational read-during-write conflicts.

3. Reset Strategy:

- *Proposed:* Active-low asynchronous reset inputs (`wrst_n`, `rrst_n`) synchronized locally via standard reset synchronizers to guarantee glitch-free assertion and deterministic, synchronous recovery/removal.

Implementation Execution Plan

Step 1: Core Synchronizer & Utilities	-->	<code>rtl/cdc_sync.sv</code> , <code>rtl/bin2gray.sv</code> , <code>rtl/gray2bin.sv</code>
Step 2: Pointer & Control Engines	-->	<code>rtl/wptr_full.sv</code> , <code>rtl/rptr_empty.sv</code>
Step 3: Memory Array & Top Wrapper	-->	<code>rtl/fifo_mem.sv</code> , <code>rtl/async_fifo.sv</code>
Step 4: Formal Assertions (SVA)	-->	<code>tb/sva/async_fifo_sva.sv</code> , <code>tb/sva/gray_chk_sva.sv</code>
Step 5: Verification Harness & TB	-->	<code>tb/pkg/*.sv</code> , <code>tb/tb_async_fifo.sv</code>
Step 6: Fault Injection Implementations	-->	<code>fault_injection/*.sv</code> , <code>tb/tb_fault_injector.sv</code>
Step 7: Automated Scripts & Synthesis	-->	<code>sim/Makefile</code> , <code>synth/synth_yosys.tcl</code> , <code>scripts/*.py</code>
Step 8: Final Reports & Documentation	-->	<code>docs/cdc_analysis.md</code> , <code>docs/cdc_checklist.md</code> , <code>README.md</code>
